# Roteiro de fala — Defesa do TCC

Autosim: DSL para o Ensino de Autômatos Finitos

Tempo planejado: **13:37** · Limite: **15 minutos** · 36 slides

**Como usar:** a coluna *Tempo* traz o tempo do slide e, abaixo, o tempo acumulado ao final dele. Use o acumulado para conferir o ritmo durante a fala. Os tempos já incluem as trocas de slide e as pausas, e sobram cerca de 1:20 para imprevistos. Nos slides de *Roteiro* do início de cada seção, diga a frase de transição, que cita o capítulo da monografia para a banca acompanhar, e avance.

**Demonstração (slide 28):** antes da defesa, deixe a Autosim aberta em uma janela com `examples/example1.asl` e um terminal na pasta da ferramenta. No slide 28, alterne para a Autosim (Alt+Tab), mostre e volte para os slides. Se a demonstração ao vivo falhar, abra `video/demo-autosim.mp4` (1min44s) no reprodutor de vídeo.

| Slide | Tempo | Fala |
|---|---|---|
| **Abertura** | | **0:27** |
| 1 — Capa | **0:15**<br>até 0:15 | Boa noite a todos. Somos João Thomaz Vieira e Klaus Siegfried Beckmann e vamos apresentar a Autosim, uma ferramenta baseada em DSL para o ensino de autômatos finitos, orientada pelo professor Diógenes Furlan. |
| 2 — Roteiro | **0:12**<br>até 0:27 | A apresentação segue os nove capítulos da monografia, na mesma ordem e com a mesma numeração, para que vocês possam acompanhar pelo texto. |
| **1. Introdução** | | **1:48** |
| 3 — Roteiro | **0:03**<br>até 0:30 | Começando pelo Capítulo 1, a introdução. |
| 4 — Contexto e problema | **0:45**<br>até 1:15 | Autômatos finitos são um pilar da computação, mas a disciplina é conhecida por ser difícil. Morazán e Antunez apontam o porquê: na programação o aluno escreve, executa e vê o resultado; em autômatos, ele projeta no papel, sem experimentar nem receber retorno imediato. As ferramentas mais usadas, como o JFLAP, são só visuais: não há texto para versionar e nem diagnóstico do erro. Já a FSM descreve autômatos como código, mas não os desenha. A lacuna é unir as três coisas: texto, diagnóstico e simulação visual. |
| 5 — Pergunta e proposta | **0:35**<br>até 1:50 | A pergunta que orienta o trabalho é: como uma DSL textual, processada por um compilador com diagnósticos localizados e acoplada a uma simulação visual, pode oferecer retroalimentação imediata ao estudante? Nossa resposta é a Autosim: o aluno escreve o autômato em uma DSL em português, o compilador aponta os erros com linha e coluna, e a interface desenha e anima o autômato automaticamente. |
| 6 — Objetivos | **0:25**<br>até 2:15 | Para isso, projetamos a DSL, implementamos o compilador, desenvolvemos o simulador, construímos a interface e validamos tudo. Um ponto importante: a validação é funcional. Medir o efeito sobre a aprendizagem exigiria um estudo com turmas, que fica como trabalho futuro. |
| **2. Fundamentação Teórica** | | **1:13** |
| 7 — Roteiro | **0:03**<br>até 2:18 | Capítulo 2, a fundamentação teórica. |

| Slide | Tempo | Fala |
|---|---|---|
| 8 — Autômatos finitos | **0:40** até 2:58 | Um AFD é uma quíntupla: estados, alfabeto, transições, estado inicial e estados finais. No AFN, uma transição pode levar a vários estados e pode haver transições vazias, as épsilon. Os dois são equivalentes e reconhecem as linguagens regulares. Este autômato aceita as cadeias que contêm `ab`: aceita `ab` e `aab`, rejeita `ba` e a cadeia vazia. Ele vai reaparecer na DSL e na interface. |
| 9 — Compiladores e DSL | **0:30** até 3:28 | A Autosim tem um compilador, com as três fases de análise: a léxica forma os *tokens*, a sintática monta a árvore sintática, e a semântica verifica o que a gramática não captura, como referências a estados não declarados. Uma DSL, segundo Fowler, é uma linguagem de expressividade limitada a um domínio. A FSM é interna, embutida em Racket; a Autosim é externa, com sintaxe e compilador próprios. |
| **3. Trabalhos Relacionados** | **1:03** | |
| 10 — Roteiro | **0:03** até 3:31 | Capítulo 3, os trabalhos relacionados. |
| 11 — Panorama | **0:35** até 4:06 | Analisamos cinco trabalhos. Quatro são gráficos: o JFLAP, referência mundial; o GAM, limitado a dez estados; o simulador da UFSC, que roda na web; e o AutomatoGraph, que gera a gramática regular. O quinto, a FSM, é o mais próximo do nosso: é textual, permite testar e versionar, mas a execução é só em texto, e a visualização ficou como trabalho futuro dos próprios autores. |
| 12 — Análise comparativa | **0:25** até 4:31 | Lado a lado: as ferramentas gráficas visualizam, mas não têm texto nem diagnóstico; a FSM tem texto, mas não visualiza. A Autosim reúne as duas coisas e acrescenta o diagnóstico de um compilador. |
| **4. Metodologia de Construção da Solução** | **0:43** | |
| 13 — Roteiro | **0:03** até 4:34 | Capítulo 4, a metodologia. |
| 14 — Metodologia | **0:40** até 5:14 | É uma pesquisa aplicada, que combina pesquisa bibliográfica com o desenvolvimento de um artefato. O trabalho seguiu seis etapas, de março a novembro, e a ferramenta foi construída na ordem do próprio pipeline: léxico, sintático e semântico, depois simulador e interface. Na revisão final, conferimos texto, especificação e código entre si, e as divergências foram corrigidas na própria ferramenta, nas versões 1.0.1 a 1.0.4. A validação usou três instrumentos, cada um com seu critério de aceitação. |
| **5. Ferramentas para Implementação** | **0:38** | |
| 15 — Roteiro | **0:03** até 5:17 | Capítulo 5, as ferramentas. |
| 16 — Por que Rust | **0:35** até 5:52 | Escolhemos Rust por quatro requisitos: gera um binário único, sem instalar JVM ou interpretador; as enumerações com *match* exaustivo garantem que cada fase trate todos os casos; há bibliotecas próprias para compiladores com bons diagnósticos; e há interface multiplataforma. Usamos Logos no léxico, Chumsky no sintático, Ariadne nos diagnósticos e Iced na interface. Os pacotes para os três sistemas são gerados pelo GitHub Actions. |
| **6. Projeto da DSL** | **1:03** | |

| Slide | Tempo | Fala |
|---|---|---|
| 17 — Roteiro | **0:03** até 5:55 | Capítulo 6, o projeto da linguagem. |
| 18 — A linguagem | **0:30** até 6:25 | Este é o mesmo autômato do exemplo, agora em texto. Cada elemento da quíntupla tem a sua declaração: alfabeto, estados, inicial, finais e transições. No fim, o comando `simular` testa as cadeias: `ab` é aceita e `bbb` é rejeitada. Por ser texto puro, o arquivo pode ser versionado com Git. |
| 19 — Gramática e decisões | **0:30** até 6:55 | A linguagem foi formalizada em EBNF; aqui está um trecho, e a gramática completa está na Seção 6.2. As principais decisões: um alfabeto por arquivo; o tipo AFD ou AFN explícito, para exigir determinismo só do AFD; palavras-chave em português, sem diferenciar maiúsculas; e suporte a transições épsilon. |
| **7. Implementação** | | **1:58** |
| 20 — Roteiro | **0:03** até 6:58 | Capítulo 7, a implementação. |
| 21 — Pipeline | **0:25** até 7:23 | O arquivo passa pelo léxico, pelo sintático e pelo semântico, e então segue para o simulador e a interface. Se qualquer fase encontra um erro, a execução para ali e o diagnóstico é exibido. Um detalhe curioso: o Logos gera um AFD para ler o texto, ou seja, é um autômato lendo a linguagem de autômatos. |
| 22 — Semântica e diagnósticos | **0:30** até 7:53 | A análise semântica faz três verificações: duplicatas, referências a estados, símbolos ou autômatos não declarados e, só no AFD, determinismo. Os erros são acumulados, então o aluno vê todos de uma vez. À direita, um diagnóstico real: o arquivo, a linha e a coluna, o trecho com o erro realçado e a descrição em português. |
| 23 — Simulador | **0:30** até 8:23 | No AFN, acender todos os estados ativos ao mesmo tempo deixava a execução difícil de acompanhar. Por isso a Autosim mostra um caminho único: o aceitador, se a cadeia é aceita, ou o que chegou mais perto, se é rejeitada. O caminho é escolhido por busca em largura, então o veredito é sempre o da definição formal, e o histórico permite avançar e voltar. |
| 24 — Interface gráfica | **0:10** até 8:33 | *(Mostrar e avançar.)* Esta é a interface: o diagrama é desenhado a partir do texto, e os controles ficam embaixo. |
| 25 — Aceitação e rejeição | **0:10** até 8:43 | *(Mostrar e avançar.)* Cadeia aceita, a tela fica verde; rejeitada, fica vermelha. |
| 26 — AFN-$\varepsilon$ e diagrama grande | **0:10** até 8:53 | *(Mostrar e avançar.)* Um AFN com transição épsilon e um autômato grande, com arrastar e zoom. Se a banca quiser detalhes, voltamos a estes slides nas perguntas. |
| **8. Resultados e Discussões** | | **3:33** |
| 27 — Roteiro | **0:03** até 8:56 | Capítulo 8, resultados e discussões. Começamos mostrando a ferramenta funcionando. |
| 28 — Demonstração da aplicação | **1:30** até 10:26 | *(Alt+Tab para a Autosim.)* Com o `example1.asl` aberto: simular uma cadeia aceita passo a passo e uma rejeitada; rodar um arquivo com erro, como `sema_nondeterministic.asl`, e mostrar o diagnóstico no terminal; se der tempo, abrir `afn_epsilon.asl`. *(Alt+Tab de volta e avançar.)* |

| Slide | Tempo | Fala |
|---|---|---|
| 29 — Testes e cenários de erro | **0:25** até 10:51 | A ferramenta tem 51 testes automatizados, distribuídos pelas quatro partes, todos aprovados. Nos seis arquivos com erros propositais, cada erro foi detectado na fase certa e localizado; um arquivo com cinco erros semânticos gerou as cinco mensagens de uma só vez. |
| 30 — Validação diferencial | **0:35** até 11:26 | Para o simulador, fizemos uma validação diferencial: todas as cadeias até o comprimento 12, em oito autômatos clássicos, foram simuladas pela Autosim e comparadas com a definição da linguagem e com um simulador de referência independente. Foram 57.368 cadeias, sem nenhuma divergência. Um AFN e o AFD equivalente aceitaram exatamente as mesmas cadeias. O limite é que a verificação é exaustiva só até esse comprimento. |
| 31 — Avaliação preliminar | **0:30** até 11:56 | *(Completar com os resultados do formulário.)* Depois da entrega do texto, aplicamos um formulário com estudantes, usando o guia rápido da ferramenta. Ele não faz parte da monografia e não mede aprendizagem, mas indica como a Autosim foi recebida: . . . |
| 32 — Discussão | **0:30** até 12:26 | Voltando aos objetivos: os cinco foram atendidos, cada um com uma evidência. A interface foi demonstrada, mas não passou por testes automatizados. E o limite mais importante: mostramos que a retroalimentação é produzida corretamente, não que ela melhora a aprendizagem. |
| **9. Conclusão** | | **1:11** |
| 33 — Roteiro | **0:03** até 12:29 | Capítulo 9, para concluir. |
| 34 — Contribuições | **0:30** até 12:59 | Respondendo à pergunta de pesquisa no plano funcional: a Autosim oferece retroalimentação imediata na construção e na execução de autômatos. O trabalho entrega a DSL em português, o compilador com diagnósticos localizados, o simulador de caminho único e a interface animada, validados por testes e por mais de 57 mil simulações, e distribuídos para os três sistemas operacionais. |
| 35 — Limitações e trabalhos futuros | **0:30** até 13:29 | Como limitações: um alfabeto por arquivo, sem modo de linha de comando, sem editor embutido e sem conversão de AFN para AFD. Como trabalhos futuros, queremos tratar esses pontos, estender a ferramenta a autômatos com pilha e máquinas de Turing e fazer estudos controlados com estudantes. |
| 36 — Obrigado | **0:08** até 13:37 | Obrigado pela atenção. Ficamos à disposição para as perguntas da banca. |